

# объектно-ориентированное программирование

структурные паттерны

паттерны?

# что такое паттерны

## паттерны?

- паттерны – устоявшиеся шаблоны для решения определённых проблем проектирования
  - термин используется и вне ООП
  - в контексте ООП будем говорить про паттерны GoF
- проблема проектирования – какая-то задача по моделированию структуры/логики

# зачем нужны паттерны

## паттерны?

- стандартные решения определённых проблем
  - “единый язык” дизайна ПО, проще понимать при командной разработке
  - по выбранному паттерну можно определить решаемую проблему
- зачем изучаем
  - познакомиться со стандартными проблемами при проектировании объектной модели
  - познакомиться со стандартными решениями этих проблем

# шаблон ≠ калька

## паттерны?

- важно понимать:
  - паттерн – отправная точка, референс
  - использовать паттерн ≠ написать 1в1 как показали на лекции/в интернете
- конкретная бизнес логика “уникальна”
  - реализация паттерна может отходить от “сухого стандарта” если того требует бизнес логика
  - паттерны вполне можно комбинировать друг с другом

# проблематика паттерны?

- ООП – это про взаимодействие объектов
  - мы выделяем какие-то сущности в системе
  - мы выделяем какую-то логику в системе
- как сделать так чтобы эта логика превратилась в адекватную объектную модель?
  - принципы SOLID как пререквизит к применению паттернов
  - ориентируясь на принципы SOLID понимаем что есть проблема в реализации, используем паттерны чтобы её решить

# решение

## паттерны?

- использовать концепции ООП
  - [де]композиция
  - полиморфизм
- паттерны по сути – шаблоны использования композиции и полиморфизма

# структурные паттерны

## паттерны?

- структурные паттерны – решают проблему структурирования логики
  - “как мне разделить всё это на классы, чтобы было хорошо?”

адаптер

# проблематика

## адаптер

- я хочу использовать объект одного типа в каком-то месте, но это кажется не правильным

# проблематика (кейс 1)

## адаптер

- существуют бизнесовая и инфраструктурная реализации
- инфраструктурная реализация нам не подконтрольна (идёт из библиотеки)

# проблематика (кейс 1)

## адаптер

```
public class SentryClient
{
    public void SaveLog(
        string message,
        DateTimeOffset timestamp,
        int severity)
    {
        ...
    }
}
```

```
public class SomeService(
    SentryClient sentryClient)
{
    public void DoOperation()
    {
        sentryClient.SaveLog(
            "Started operation",
            DateTimeOffset.UtcNow,
            severity: 1);
    }
}
```

# проблематика (кейс 1)

## адаптер

- если сменится провайдер логов – нужно менять бизнес логику
- детали инфраструктурной реализации протекают в клиент
- нарушение DIP
- но мы не можем реализовать интерфейс в GraylogClient

# проблематика (кейс 2)

## адаптер

- существуют бизнесовая(1) и бизнесовая(2) реализации
- реализации (1) нужно использовать реализацию (2)
- реализации (1) может быть нужно использовать другие реализации
- реализация (2) не должна знать про реализацию (1)

# проблематика (кейс 2)

## адаптер

```
public interface IAdressee
{
    void ReceiveMessage(string message);
}
```

```
public class Messenger(
    IReadOnlyCollection<IAdressee> adressees)
{
    public void SendMessage(string message)
    {
        foreach (IAdressee adressee in adressees)
        {
            adressee.ReceiveMessage(message);
        }
    }
}
```

# проблематика (кейс 2)

## адаптер

```
public class User
{
    private readonly List<string> _receivedMessages = [];

    public void StoreMessage(string message)
    {
        _receivedMessages.Add(message);
    }
}
```

# проблематика (кейс 2)

## адаптер

- мы хотим использовать `User` в качестве `IAddressee`
- метод `User.StoreMessage` может вызываться в других местах
- `User` не должен знать про `Messenger` и его абстракции
- поэтому мы не можем просто реализовать `IAddressee` в `User` – нарушение SRP

# проблематика

## адаптер

- у нас есть объект одного типа, который мы хотим использовать как объект другого типа
- но по определённым причинам не можем

промежуточный тип, использующий объект  
одного типа, для реализации интерфейса  
другого типа



**адаптер**

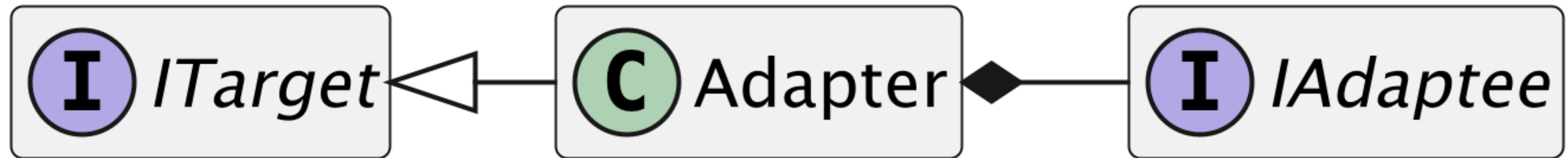
# структура

## адаптер

- target  
целевой интерфейс, через который мы хотим взаимодействовать с объектом, изначально его не реализующим
- adaptee  
адаптируемый тип
- adapter  
тип-обёртка, реализует целевой интерфейс, содержит объект адаптируемого типа, перенаправляет в него вызовы поведений целевого интерфейса

# схема

## адаптер



не обязательно интерфейс

# пример использования (кейс 1)

## адаптер



# пример использования (кейс 1)

## адаптер

```
public class SomeService(  
    ILogger logger)  
{  
    public void DoOperation()  
    {  
        logger.LogInformation("Started operation");  
    }  
}
```

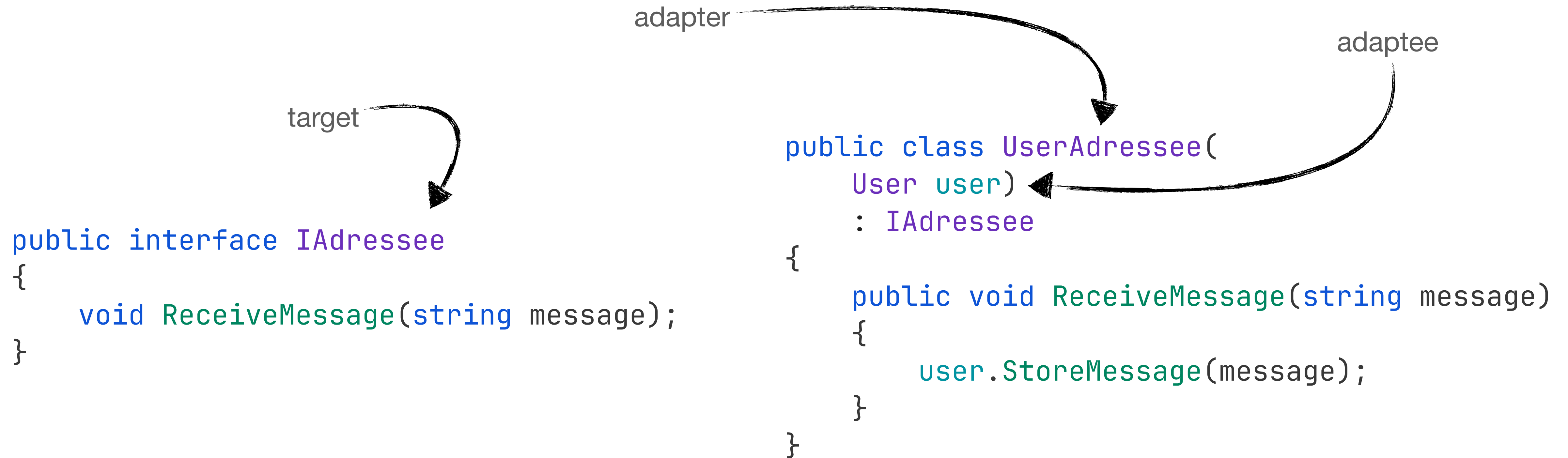
# пример использования (кейс 1)

## адаптер

- бизнесовая реализация зависит от абстракции – соблюдаем DIP
- можем безболезненно заменить реализацию – OCP/protected variations

# пример использования (кейс 2)

## адаптер



# пример использования (кейс 2)

## адаптер

- User не связан с абстракциями Messenger – соблюдаем SRP
- при добавлении новых интеграций User с другими частями объектной модели – делаем новые адаптеры
  - тип User не засоряется связями с другими абстракциями
  - тип User необходимо тестировать только в контексте его функционала

**МОСТ**

# проблематика

## МОСТ

- у меня есть составная логика
  - можно выделить несколько уровней реализации этой логики
  - каждый уровень может варьироваться
  - вариации на разных уровнях могут комбинироваться между собой

# проблематика (кейс)

## МОСТ

- существует программа, которая на определённом пиксельном холсте рисует фигуры
- фигуры могут быть разными (круг, квадрат, ...)
- холсты могут быть разными (консоль, PNG, JPEG, ...)

# наивная реализация 1

## МОСТ

```
public interface IShape
{
    void Render();
}
```

```
public class ConsoleCircle : IShape
{
    public void Render()
    {
        ...
    }
}
```

```
public class PngCircle : IShape
{
    public void Render()
    {
        ...
    }
}
```

```
public class JpegCirlce : IShape
{
    public void Render()
    {
        ...
    }
}
```

# наивная реализация 1

## МОСТ

- комбинаторный взрыв – каждая комбинация отдельный класс
- дубликация всей логики
  - и отрисовки фигуры
  - и вывода на “холст”
- нет переиспользования
  - проще допустить ошибку
  - обновление логики – внесение одинаковых изменений в разные классы
- нарушаем OCP

# Наивная реализация 2

## МОСТ

```
public enum ShapeType
{
    Circle,
    Square,
}
```

```
public class ConsoleShape(
    ShapeType type)
    : IShape
{
    public void Render()
    {
        switch (type)
        {
            ...
        }
    }
}
```

# наивная реализация 2`

## МОСТ

```
public enum OutputType
{
    Console,
    Png,
    Jpeg,
}
```

```
public class Circle(
    OutputType type)
    : IShape
{
    public void Render()
    {
        switch (type)
        {
            ...
        }
    }
}
```

# Наивная реализация 2"

## МОСТ

```
public abstract class Circle : IShape
{
    public abstract void Render();

    protected int[,] CalculatePixels() { ... }
}
```

```
public class ConsoleCircle : Circle
{
    public override void Render()
    {
        int[,] pixels = CalculatePixels();
        ...
    }
}
```

```
public class PngCircle : Circle
{
    public override void Render()
    {
        int[,] pixels = CalculatePixels();
        ...
    }
}
```

```
public class JpegCircle : Circle
{
    public override void Render()
    {
        int[,] pixels = CalculatePixels();
        ...
    }
}
```

# наивная реализация 2\*

## МОСТ

- комбинаторный взрыв есть, но часть логики переиспользуется
- часть логики всё ещё дублируется
- всё ещё нарушаем OCP
- как понять что переиспользовать, а что дублировать?
  - ответ – переиспользовать всё

разделение объектной модели на абстракции разных уровней  
реализации абстракций более высокого уровня,  
использующие абстракции более низкого уровня и являются  
“МОСТОМ”



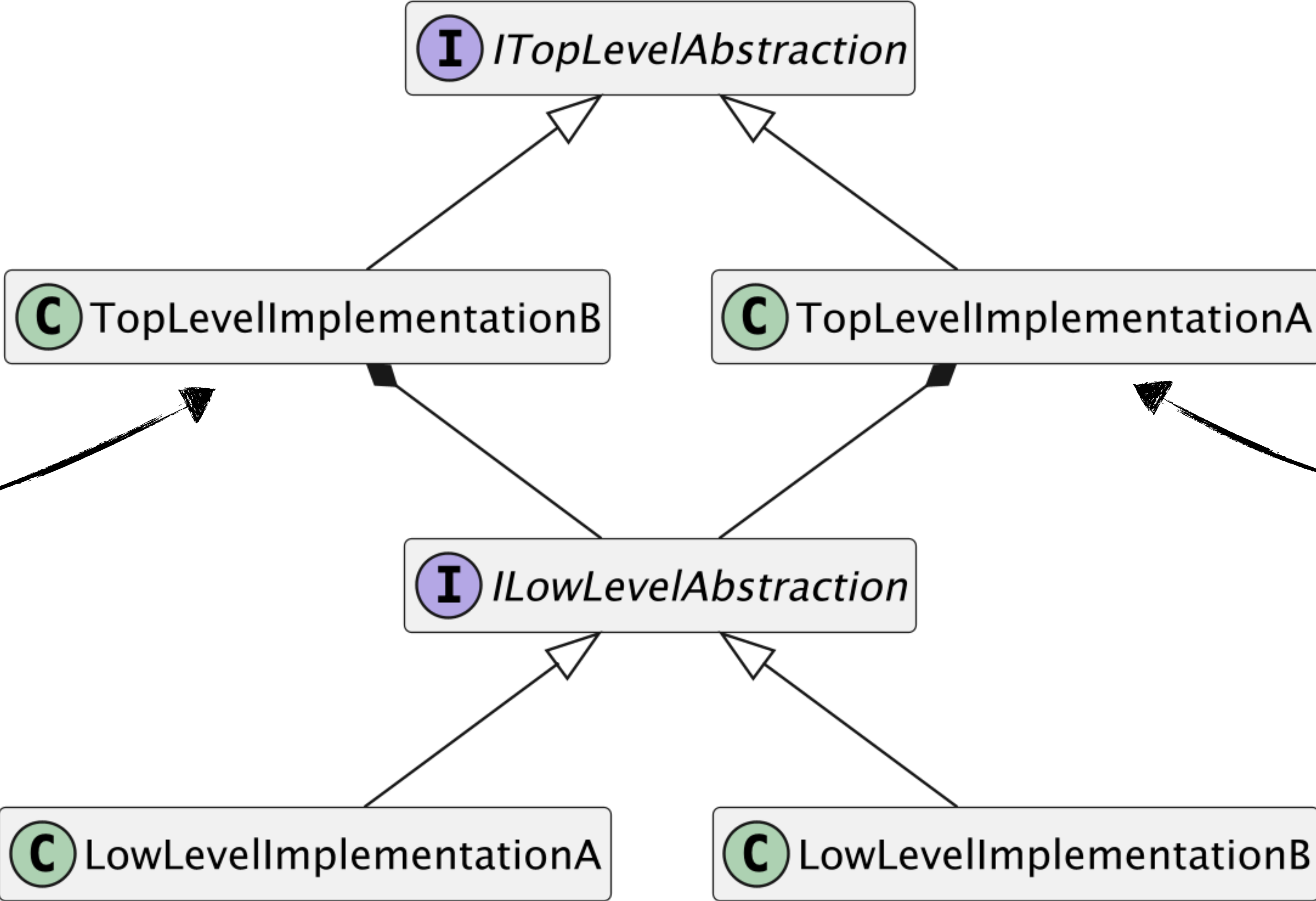
**МОСТ**

cxema

MOCT

MOCT

MOCT



# пример использования

## МОСТ

top-level abstraction

```
public interface IShape
{
    void Render(IRenderingEngine engine);
}
```

low-level abstraction

```
public interface IRenderingEngine
{
    void RenderPixels(int[,] pixels);
}
```

# пример использования

## МОСТ

top-level implementations

```
public class Circle : IShape
{
    public void Render(
        IRenderingEngine engine)
    {
        ...
    }
}
```

```
public class Square : IShape
{
    public void Render(
        IRenderingEngine engine)
    {
        ...
    }
}
```

# пример использования

## МОСТ

low-level implementations

```
public class ConsoleRenderingEngine
    : IRenderingEngine
{
    public void RenderPixels(
        int[,] pixels)
    {
        ...
    }
}
```

```
public class PngRenderingEngine(
    string fileName)
    : IRenderingEngine
{
    public void RenderPixels(
        int[,] pixels)
    {
        ...
    }
}
```

# пример использования

## МОСТ

```
var circle = new Circle();  
var square = new Square();
```

```
var consoleEngine = new ConsoleRenderingEngine();  
var pngEngine = new PngRenderingEngine("shape.png");  
var jpegEngine = new JpegRenderingEngine("shape.jpg");
```

```
circle.Render(consoleEngine);  
circle.Render(pngEngine);  
circle.Render(jpegEngine);
```

```
square.Render(consoleEngine);  
square.Render(pngEngine);  
square.Render(jpegEngine);
```

# пример использования

## мост

- в указанном примере не использовалась композиция
  - это не значит что она не может быть использована
  - это не значит что использование композиции  $\neq$  мост
- мост – сама идея декомпозиции логики на абстракции разного уровня; остальное – детали реализации

# разбиение на другие принципы

## мост

- способ реализации OCP
- способ реализации protected variations
- можно назвать подвидом адаптера
  - мост больше уходит в полиморфизм для реализации и OCP
  - адаптер больше уходит в изоляцию логики и SRP

**КОМПОНОВЩИК**

# проблематика

## КОМПОНОВЩИК

- у меня есть составная/иерархичная структура данных
  - есть логика, выполняемая над каждым элементом
  - для разных элементов логика может отличаться

# проблематика (кейс)

## КОМПОНОВЩИК

- разработка photoshop-подобного графического редактора
- есть разные элементы (круг, квадрат, ...)
- есть группы элементов
- все элементы можно передвигать
  - при передвигании группы-элементов, передвигание должно транслироваться дочерним элементам

# Наивная реализация

## КОМПОНОВЩИК

```
public interface IGraphicComponent
{
    Position Position { get; set; }
}
```

```
public class Circle : IGraphicComponent
{
    public Position Position { get; set; }
}
```

```
public class Square : IGraphicComponent
{
    public Position Position { get; set; }
}
```

```
public class ComponentGroup : IGraphicComponent
{
    public Position Position { get; set; }

    public List<IGraphicComponent> Components { get; } = [];
}
```

# Наивная реализация

## КОМПОНОВЩИК

```
void MoveBy(IGraphicComponent component, int x, int y)
{
    if (component is Circle circle)
    {
        circle.Position = new Position(
            circle.Position.X + x,
            circle.Position.Y + y);
    }

    if (component is Square square)
    {
        square.Position = new Position(
            square.Position.X + x,
            square.Position.Y + y);
    }

    if (component is ComponentGroup group)
    {
        foreach (IGraphicComponent child in group.Components)
        {
            MoveBy(child, x, y);
        }
    }
}
```

# наивная реализация

## КОМПОНОВЩИК

- нарушение OCP
  - костыли над полиморфизмом
- нарушение LSP и DIP
  - свойство `Position` у `ComponentGroup` – фиктивное

представление древовидной структуры  
объектов в виде одного композитного объекта

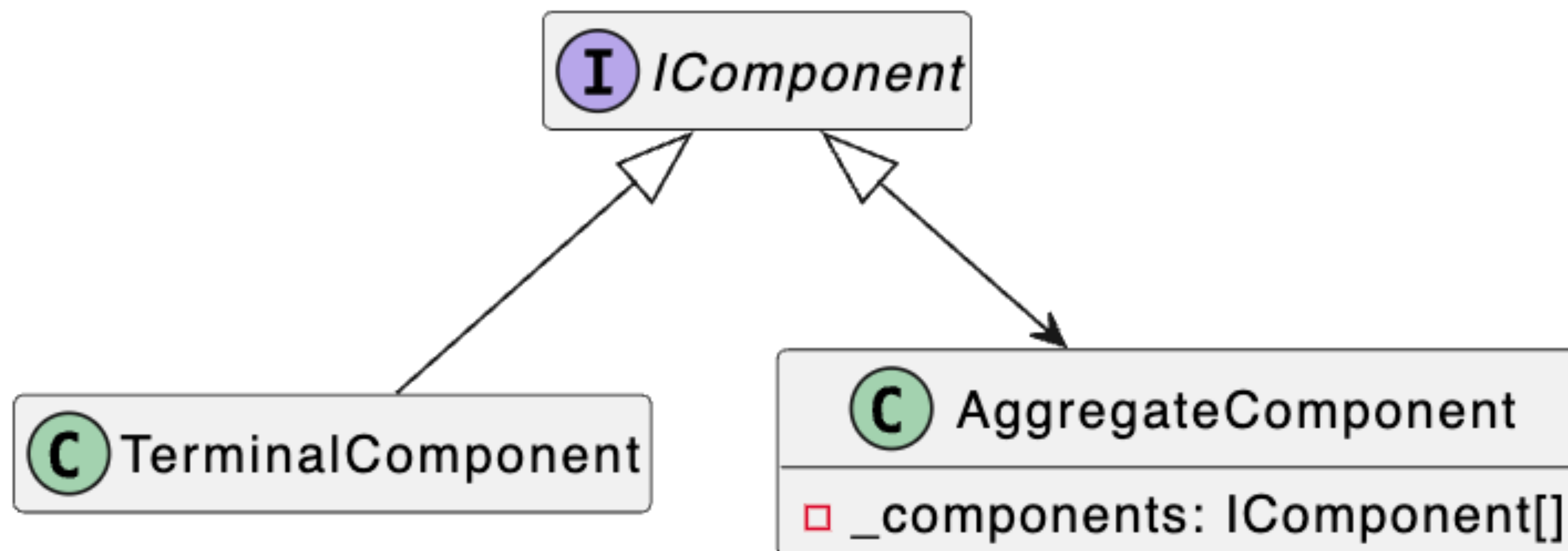


**КОМПОНОВЩИК**

# схема

## КОМПОНОВЩИК

- терминальные узлы – не содержат другие узлы
- агрегатные узлы – содержат другие узлы



# ПО СУТИ...

## КОМПОНОВЩИК

- очень конкретный юзкейс полиморфизма
- но, достаточно часто встречаемый, поэтому вынесен в отдельный паттерн
- МОЖНО ВЫДЕЛИТЬ
  - обязательность отношения целое/часть
  - возможность работы с составными объектами как с единым целым

# пример использования

## КОМПОНОВЩИК

```
public interface IGraphicComponent
{
    void MoveBy(int x, int y);
}
```

# пример использования

## КОМПОНОВЩИК

```
public class Circle(  
    Position position)  
    : IGraphicComponent  
{  
    public Position Position { get; private set; } = position;  
  
    public void MoveBy(int x, int y)  
    {  
        Position = Position.MovedBy(x, y);  
    }  
}
```

```
public class Square(  
    Position position)  
    : IGraphicComponent  
{  
    public Position Position { get; private set; } = position;  
  
    public void MoveBy(int x, int y)  
    {  
        Position = Position.MovedBy(x, y);  
    }  
}
```

# пример использования

## КОМПОНОВЩИК

```
public class ComponentGroup(  
    IEnumerable<IGraphicComponent> components)  
    : IGraphicComponent  
{  
    private readonly List<IGraphicComponent> _components = [.. components];  
  
    public void MoveBy(int x, int y)  
    {  
        foreach (IGraphicComponent component in _components)  
        {  
            component.MoveBy(x, y);  
        }  
    }  
}
```

# пример использования

## КОМПОНОВЩИК

```
var circle = new Circle();  
circle.MoveBy(1, 2);
```

```
var square = new Square();  
square.MoveBy(10, 0);
```

```
var composite = new GraphicComponentGroup([circle, square]);  
composite.Draw();  
Console.WriteLine("----");
```

```
composite.MoveBy(1, 1);  
composite.Draw();
```

```
Circle: 1, 2  
Square: 10, 0  
----  
Circle: 2, 3  
Square: 11, 1
```

декоратор

# проблематика

## декоратор

- существует тип/набор типов
  - необходимо добавить к их объектам какое-то поведение

# проблематика (кейс)

## декоратор

- разработка кассовой системы
  - существуют штучные товары
  - существуют групповые упаковки товаров
  - необходимо иметь возможность получить стоимость товара
  - необходимо иметь возможность применять скидки товарам

# проблематика (кейс)

## декоратор

```
public interface IProduct
{
    decimal CalculateCost();
}
```

```
public class SingularItem(
    decimal price)
    : IProduct
{
    public decimal CalculateCost()
    {
        return price;
    }
}

public class GroupPackageItems(
    IReadOnlyCollection<IProduct> products)
    : IProduct
{
    public decimal CalculateCost()
    {
        return products
            .Sum(product => product.CalculateCost());
    }
}
```

# Наивная реализация декоратор

```
public class SingularItem(  
    decimal price,  
    decimal discount)  
    : IProduct  
{  
    public decimal CalculateCost()  
    {  
        return price * (1 - discount);  
    }  
}
```

```
public class GroupPackageItems(  
    IReadOnlyCollection<IProduct> products,  
    decimal discount)  
    : IProduct  
{  
    public decimal CalculateCost()  
    {  
        decimal cost = products  
            .Sum(product => product.CalculateCost());  
  
        return cost * (1 - discount);  
    }  
}
```

# наивная реализация

## декоратор

- нарушение SRP
  - классы ответственны и за свою логику, и за логику расчёта скидки
- нарушение OCP
  - если логика расчёта скидки изменится, нужно изменить все реализации
- дубликация логики

# наивная реализация` декоратор

```
public class DiscountedSingularItem(  
    decimal price,  
    decimal discount)  
    : SingularItem(price)  
{  
    public override decimal CalculateCost()  
    {  
        return base.CalculateCost() * (1 - discount);  
    }  
}
```

```
public class DiscountedGroupPackageItems(  
    IReadOnlyCollection<IProduct> products,  
    decimal discount)  
    : GroupPackageItems(products)  
{  
    public override decimal CalculateCost()  
    {  
        return base.CalculateCost() * (1 - discount);  
    }  
}
```

# наивная реализация`

## декоратор

- нарушение SRP
  - Хоть логика и в отдельных классах, но объект-наследник имеет две ответственности
- комбинаторный взрыв наследников
  - представьте что может появиться ещё один модификатор – будет декартово произведение
- сильная связанность наследников
  - изменится конструктор базового класса – нужно менять дочерний
- дубликация логики

тип-обёртка, дополняющий логику объекта,  
который реализует ту же абстракцию



декоратор

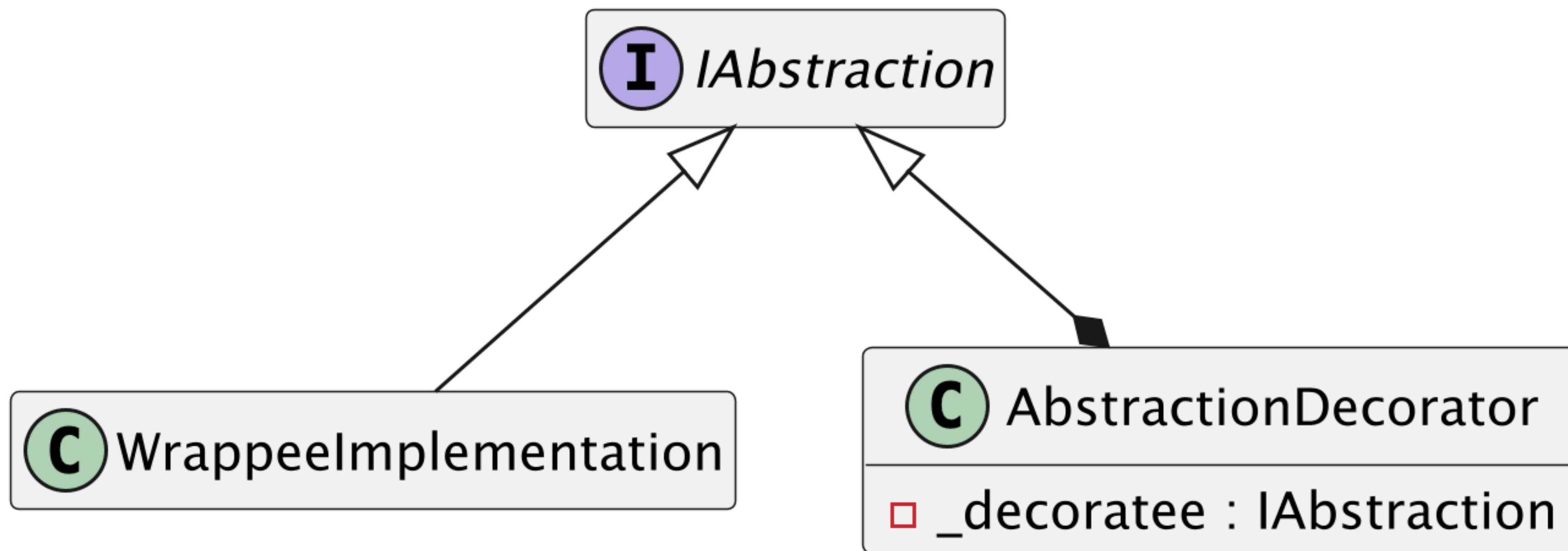
# структура

## декоратор

- абстракция  
какой-либо интерфейс, определяющий поведения
- декоратор  
тип, реализующий абстракцию, содержащий объект данной абстракции
- decoratee  
объект, типа, реализующего абстракцию, оборачиваемый в декоратор

# схема

## декоратор



# пример использования декоратор

```
public class DiscountedProduct(  
    IProduct product,  
    decimal discount)  
    : IProduct  
{  
    public decimal CalculateCost()  
    {  
        return product.CalculateCost() * (1 - discount);  
    }  
}
```

# пример использования декоратор

```
Console.WriteLine(singular.CalculateCost());  
Console.WriteLine(group.CalculateCost());
```

```
Console.WriteLine("-----");
```

```
Console.WriteLine(new DiscountedProduct(singular, 0.2m).CalculateCost());  
Console.WriteLine(new DiscountedProduct(group, 0.1m).CalculateCost());
```

1

3

-----

0,8

2,7

прокси

# проблематика

## прокси

- существует тип/набор типов
  - необходимо добавить пред-условия выполнения их операций
- (да, похоже на декоратор, но не декоратор)

# проблематика (кейс)

## прокси

- необходимо реализовать класс для получения конфигураций
  - загрузка конфигураций должна быть ленивой
  - необходимо ограничивать доступ к объектам конфигурации
  - необходимо кешировать объекты конфигураций

# наивная реализация прокси

```
public class JsonDataStore
{
    private readonly IReadOnlyDictionary<string, JsonDocument> _values;

    public JsonDataStore(string filePath)
    {
        string content = File.ReadAllText(filePath);
        _values = JsonSerializer.Deserialize<Dictionary<string, JsonDocument>>(content) ?? [];
    }
}
```

# Наивная реализация прокси

```
public class JsonDataStore
{
    private readonly IReadOnlyDictionary<string, JsonDocument> _values;

    public JsonDataStore(string filePath)
    {
        string content = File.ReadAllText(filePath);
        _values = JsonSerializer.Deserialize<Dictionary<string, JsonDocument>>(content) ?? [];
    }

    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        if (_values.TryGetValue(key, out JsonDocument? serialized) is false)
        {
            value = default;
            return false;
        }

        value = JsonSerializer.Deserialize<T>(serialized);
        return value is not null;
    }
}
```

# наивная реализация прокси

```
public class JsonDataStore(string filePath)
{
    private IReadOnlyDictionary<string, JsonDocument>? _values;

    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        EnsureValuesLoaded();

        if (_values.TryGetValue(key, out JsonDocument? serialized) is false)
        {
            value = default;
            return false;
        }

        value = JsonSerializer.Deserialize<T>(serialized);
        return value is not null;
    }

    [MemberNotNull(nameof(_values))]
    private void EnsureValuesLoaded()
    {
        string content = File.ReadAllText(filePath);
        _values = JsonSerializer.Deserialize<Dictionary<string, JsonDocument>>(content) ?? [];
    }
}
```

# Наивная реализация прокси

```
public class JsonDataStore(  
    string filePath,  
    ICurrentUser currentUser)  
{  
    private IReadOnlyDictionary<string, JsonDocument>? _values;  
  
    public bool TryGetValue<T>(  
        string key,  
        [NotNullWhen(true)] out T? value)  
    {  
        if (key.Contains("secure", StringComparison.OrdinalIgnoreCase)  
            && currentUser.IsAdmin is false)  
        {  
            value = default;  
            return false;  
        }  
  
        EnsureValuesLoaded();  
  
        if (_values.TryGetValue(key, out JsonDocument? serialized) is false)  
        {  
            value = default;  
            return false;  
        }  
  
        value = JsonSerializer.Deserialize<T>(serialized);  
        return value is not null;  
    }  
  
    [MemberNotNull(nameof(_values))]  
    private void EnsureValuesLoaded()  
    {  
        string content = File.ReadAllText(filePath);  
        _values = JsonSerializer.Deserialize<Dictionary<string, JsonDocument>>(content) ?? [];  
    }  
}
```

# Наивная реализация прокси

```
public class JsonDataStore(
    string filePath,
    ICurrentUser currentUser)
{
    private readonly Dictionary<string, object?> _valuesCache = [];
    private IReadOnlyDictionary<string, JsonDocument>? _values;

    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        if (key.Contains("secure", StringComparison.OrdinalIgnoreCase)
            && currentUser.IsAdmin is false)
        {
            value = default;
            return false;
        }

        EnsureValuesLoaded();

        if (_values.TryGetValue(key, out JsonDocument? serialized) is false)
        {
            value = default;
            return false;
        }

        if (_valuesCache.TryGetValue(key, out object? untypedValue))
        {
            value = untypedValue is T typed ? typed : default;
            return value is not null;
        }

        value = JsonSerializer.Deserialize<T>(serialized);
        _valuesCache[key] = value;
        return value is not null;
    }

    [MemberNotNull(nameof(_values))]
    private void EnsureValuesLoaded()
    {
        string content = File.ReadAllText(filePath);
        _values = JsonSerializer.Deserialize<Dictionary<string, JsonDocument>>(content) ?? [];
    }
}
```

# наивная реализация

## прокси

- нарушение SRP
- нарушение OCP
- ну и просто... плохо...

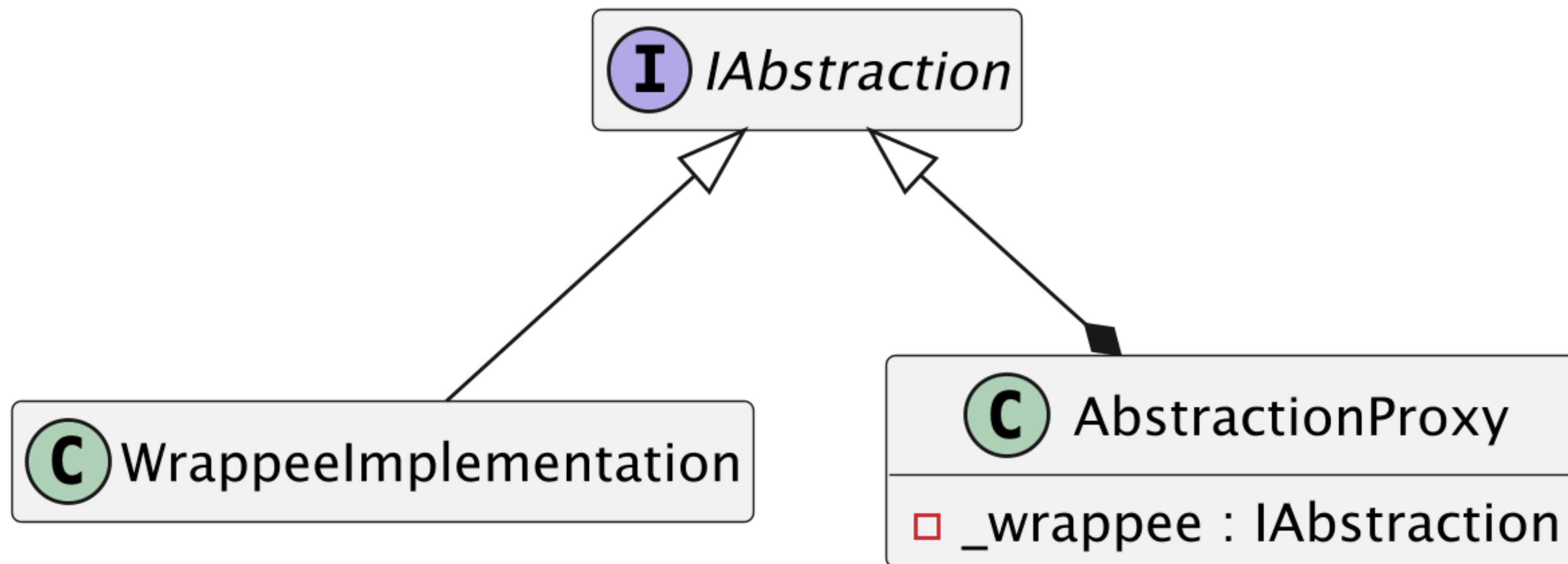
тип-обёртка, реализующий логику контроля  
доступа к объекту, который реализует ту же  
абстракцию



прокси

# схема

## прокси



# ВИДЫ прокси

```
public interface IDataStore
{
    bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value);
}
```

```
public class JsonDataStore(
    string filePath)
    : IDataStore
{
    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        ...
    }
}
```

# virtual / lazy

## прокси

```
public class LazyJsonDataStore(
    string filePath)
    : IDataStore
{
    private readonly Lazy<JsonDataStore> _dataStore =
        new(() => new JsonDataStore(filePath));

    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        return _dataStore.Value.TryGetValue(
            key,
            out value);
    }
}
```

```
public class LazyDataStore(
    Func<IDataStore> createDataStore)
    : IDataStore
{
    private readonly Lazy<IDataStore> _dataStore =
        new(createDataStore);

    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        return _dataStore.Value.TryGetValue(
            key,
            out value);
    }
}
```

# defensive

## прокси

```
public class AuthorizedDataStore(
    IDataStore dataStore,
    ICurrentUser currentUser)
    : IDataStore
{
    public bool TryGetValue<T>(
        string key,
        [NotNullWhen(true)] out T? value)
    {
        if (key.Contains("secure", StringComparison.OrdinalIgnoreCase)
            && currentUser.IsAdmin is false)
        {
            value = default;
            return false;
        }

        return dataStore.TryGetValue(key, out value);
    }
}
```

# caching

## прокси

```
public class CachingDataStore(
    IDataStore dataStore)
    : IDataStore
{
    private readonly Dictionary<string, object?> _cache = [];

    public bool TryGetValue<T>(string key, [NotNullWhen(true)] out T? value)
    {
        if (_cache.TryGetValue(key, out object? untypedValue))
        {
            value = untypedValue is T typed ? typed : default;
            return value is not null;
        }

        if (dataStore.TryGetValue(key, out value))
        {
            _cache[key] = value;
            return true;
        }

        return false;
    }
}
```

# пример использования прокси

```
IDataStore dataStore = new LazyDataStore(() => new JsonDataStore("config.json"));
dataStore = new AuthorizedDataStore(dataStore, currentUser);
dataStore = new CachingDataStore(dataStore);

if (dataStore.TryGetValue("password_secure", out string? password))
{
    Console.WriteLine($"Password = {password}");
}
else
{
    Console.WriteLine("No password found");
}
```

# decorator vs proxy

## прокси

- ВИДЫ КОМПОЗИЦИИ  
прокси – агрегация/ассоциация  
декоратор – только агрегация
- extended dispatch vs controlled dispatch  
прокси – контролирует обрачиваемый объект  
декоратор – только расширяет логику обрачиваемого объекта
- наличие обрачиваемого объекта  
прокси – может имитировать наличие объекта  
декоратор – объект должен существовать

# decorator vs proxy

прокси

```
public interface ICalculationService
{
    double DoCalculation(int seed);
}
```

# decorator vs proxy

## прокси

```
public class CachedCalculationService(
    ICalculationService service)
    : ICalculationService
{
    private readonly Dictionary<int, double> _cache = [];

    public double DoCalculation(int seed)
    {
        if (_cache.TryGetValue(seed, out double value))
        {
            return value;
        }

        value = service.DoCalculation(seed);
        _cache[seed] = value;

        return value;
    }
}
```

# decorator vs proxy

## прокси

```
public class LoggingCalculationService(  
    ICalculationService service)  
    : ICalculationService  
{  
    public double DoCalculation(int seed)  
    {  
        double value = service.DoCalculation(seed);  
        Console.WriteLine($"Calculated value = {value}");  
  
        return value;  
    }  
}
```

# decorator vs proxy

## прокси

```
public class RetryingCalculationService(  
    ICalculationService service,  
    int maxRetryCount)  
    : ICalculationService  
{  
    public double DoCalculation(int seed)  
    {  
        var retryCount = 0;  
  
        while (true)  
        {  
            try  
            {  
                return service.DoCalculation(seed);  
            }  
            catch (Exception)  
            {  
                retryCount++;  
  
                if (retryCount > maxRetryCount)  
                {  
                    throw;  
                }  
            }  
        }  
    }  
}
```

фасад

# проблематика

## фасад

- существует операция из нескольких действий
  - отдельные части этих действий не должны знать друг о друге
  - логика вызывается из нескольких мест

# проблематика (кейс)

## фасад

- разработка системы лояльности
  - при совершении покупки нужно обновить информацию
  - рассчитать бонусы
  - перевести на следующий уровень, при необходимости
  - для этого нужно достать данные из хранилища

# проблематика (кейс)

## фасад

```
public class User
{
    public required long Id { get; init; }

    public decimal BonusPoints { get; private set; }

    public decimal TotalPurchasedAmount { get; private set; }

    public void AddBonusPoints(decimal bonusPoints)
    {
        ...
    }

    public void AddPurchasedAmount(decimal amount)
    {
        ...
    }
}
```

# проблематика (кейс)

## фасад

```
public class LoyaltyProgram(  
    decimal bonusPayoutFraction,  
    decimal maxBonusPayout)  
{  
    public LoyaltyProgram ProcessUserPurchase(  
        User user,  
        decimal purchaseAmount)  
    {  
        decimal bonusPayout = Math.Min(  
            maxBonusPayout,  
            purchaseAmount * bonusPayoutFraction);  
  
        user.AddBonusPoints(bonusPayout);  
        user.AddPurchasedAmount(purchaseAmount);  
  
        decimal loyaltyGrade = Math.Round(user.TotalPurchasedAmount / 10_000);  
  
        decimal newBonusPayoutFraction = Math.Min(1m, loyaltyGrade / 100);  
        decimal newMaxBonusPayout = loyaltyGrade * 1000;  
  
        return new LoyaltyProgram(newBonusPayoutFraction, newMaxBonusPayout);  
    }  
}
```

# проблематика (кейс)

## фасад

```
public interface IUserRepository
{
    User? FindById(long userId);

    void Update(User user);
}
```

```
public interface ILoyaltyProgramRepository
{
    LoyaltyProgram? FindByUserId(long userId);

    void UpdateForUser(
        long userId,
        LoyaltyProgram loyaltyProgram);
}
```

**оркестрация одной или набора  
сложных операций в каком-либо типе**



# пример использования фасад

```
public class UserLoyaltyService(  
    IUserRepository userRepository,  
    ILoyaltyProgramRepository loyaltyProgramRepository)  
{  
    public UserPurchaseLoyaltyResult ApplyUserPurchase(long userId, decimal purchaseAmount)  
    {  
        User? user = userRepository.FindById(userId);  
  
        if (user is null)  
            return new UserPurchaseLoyaltyResult.UserNotFound();  
  
        LoyaltyProgram? loyaltyProgram = loyaltyProgramRepository.FindByUserId(userId);  
  
        if (loyaltyProgram is null)  
            return new UserPurchaseLoyaltyResult.UserNotEnrolled();  
  
        loyaltyProgram = loyaltyProgram.ProcessUserPurchase(user, purchaseAmount);  
  
        userRepository.Update(user);  
        loyaltyProgramRepository.UpdateForUser(userId, loyaltyProgram);  
  
        return new UserPurchaseLoyaltyResult.Success();  
    }  
}
```

# на что обратить внимание

## фасад

- есть риск сделать god-class
  - стоит выделять “локальные” фасады
- потеря абстракций засчёт переиспользования логики внутри фасада
  - тяжесть рефакторинга и декомпозиции

легковес

# проблематика

## легковес

- система работает с большим числом объектов
  - объекты хранят объёмные данные
  - эти объёмные данные часто повторяются между объектами

# проблематика (кейс)

## легковес

- разработка движка для компьютерной графики
  - существуют какие-то частицы
  - частица имеет координаты и модель
  - координаты уникальны для каждой частицы
  - модель для частиц общая

# Наивная реализация

## легковес

```
public record Particle(int X, int Y, byte[] Model);

public class ParticleFactory
{
    private readonly IAssetLoader _assetLoader;

    public Particle Create(string modelName)
    {
        var model = _assetLoader.Load(modelName);
        return new Particle(0, 0, model);
    }
}
```

# наивная реализация

## легковес

- плохая производительность
  - при создании каждой частицы – грузим текстуры заново
  - каждая частица хранит дубликат данных
  - создаём много частиц – долгое создание, используем много ОЗУ

**декомпозиция объектов, выделение тяжёлых и повторяющихся данных в отдельные модели для дальнейшего переиспользования**



**легковес**

# пример использования легковес

```
public record ModelData(byte[] Value);

public record Particle(int X, int Y, ModelData Model);

public class ParticleFactory
{
    private readonly IAssetLoader _assetLoader;
    private readonly Dictionary<string, ModelData> _cache;

    public Particle Create(string modelName)
    {
        var model = _cache.TryGetValue(modelName, out var data)
            ? data
            : _cache[modelName] = new ModelData(_assetLoader.Load(modelName));

        return new Particle(0, 0, model);
    }
}
```